

Your GAN is Secretly an **Energy-Based Model**

and You Should Use Discriminator Driven Latent Sampling

Tomil SHI – Alexis MICHEL

Université Paris Dauphine-PSL

2025-2026

Energy-Based Models (EBM)

Définition

On associe à chaque point x une énergie $E(x) \in \mathbb{R}$

$$p(x) = e^{-E(x)} / Z$$

Avec Z une constante de normalisation

Intuition

Basse énergie = haute probabilité.

Problème principal

Z est intractable en haute dimension $\rightarrow$ on ne peut pas l'évaluer ni échantillonner directement.

Dynamique de Langevin

Solution pour échantillonner sans calculer Z :

$$x_{i+1} = x_i - \frac{\epsilon}{2} \nabla_x E(x) + \sqrt{\epsilon} n, n \sim N(0, I)$$

Descente de gradient

On suit $-\nabla E$ pour aller vers les creux (zones de haute proba)

Bruit gaussien

Exploration pour ne pas converger vers un seul mode

Le GAN comme Energy-Based Model

Raisonnement : du discriminateur à l'EBM

$$D^*(x) = \frac{p_d(x)}{p_d(x) + p_g(x)} \rightarrow e^{d(x)} \approx p_d/p_g \rightarrow p_d(x) \approx p_g(x)e^{d(x)} \rightarrow p_d^* = p_g(x)e^{d(x)} / Z_0$$

Propriété 1 :

Si $D = D^*$ (discriminateur optimal)
alors $p_{d^*} = p_d$ (vraie distribution)

Propriété 2 :

$e^{d(x)}$ re-pondère les régions : $\uparrow$ poids si D pense que c'est réel, $\downarrow$ poids si D pense que c'est faux.

Problème 1 : MCMC en espace pixel

Haute dimension $\rightarrow$ temps de mélange énorme

Problème 2 : p_g implicite

On ne peut pas évaluer la densité de p_g directement

Théorème principal & DDLS

Théorème 1

Soit $E(z) = -\log p_0(z) - d(G(z))$

et $p_t(z) = e^{-E(z)} / Z$ sa distribution de Boltzmann latente

Alors : si $z \sim p_t$, and $x = G(z)$, then we have $x \sim p_d^*$

(et $p_{d^*} = p_d$ quand D est optimal)

Pourquoi c'est tractable ?

- $p_0(z)$ gaussienne
- $d(G(z)) = 1$ forward pass $\rightarrow$ gradient par backprop
- Dimension de $z \ll$ dimension de x (pixels)

Algorithm 1 Discriminator Langevin Sampling

Input: $N \in \mathbb{N}_+$, $\epsilon > 0$

Output: Latent code $z_N \sim p_t(z)$

Sample $z_0 \sim p_0(z)$.

for $i < N$ **do**

$n_i \sim N(0, 1)$

$z_{i+1} = z_i - \epsilon/2 \nabla_z E(z) + \sqrt{\epsilon} n_i$

$i = i + 1$

end for

Avantage clé vs DRS/MH-GAN

On guide activement la recherche dans l'espace latent (optimisation 1er ordre) plutôt que de simplement rejeter des samples.

L'implémentation (avec WGAN-GP)

Fonctions de perte

Loss D $\mathbb{E}[D(G(z))] - \mathbb{E}[D(x)] + \lambda \cdot GP$

Loss G $-\mathbb{E}[D(G(z))]$

Gradient Penalty $GP = \mathbb{E}[(\|\nabla D(\hat{x})\|_2 - 1)^2]$

λ $\lambda = 10$

Optimiseur & scheduler (train.py)

- Adam betas=(0.0, 0.9), lr=1e-4
- StepLR, step=20, $\gamma=0.5$
- On fait 3 steps de D pour 1 step de G

Gradient Penalty (utils.py)

```
alpha = torch.rand(N, 1).expand_as(x)
interpolates = alpha*x_real + (1-alpha)*x_fake

d_interp = D(interpolates)

grads = autograd.grad(
    outputs=d_interp,
    inputs=interpolates,
    create_graph=True
)[0]

GP = ((grads.norm(2,dim=1)-1)**2).mean()
d_loss = fake.mean() - real.mean() +  $\lambda$  * GP
```

Générateur G et discriminateur D

Générateur G

$z \sim \mathcal{N}(0, I)$

dim 100

Linear + LeakyReLU

100 $\rightarrow$ 256

Linear + LeakyReLU

256 $\rightarrow$ 512

Linear + LeakyReLU

512 $\rightarrow$ 1024

Linear + Tanh

1024 $\rightarrow$ 784

Discriminateur D

x (image aplatie)

dim 784

Linear + LeakyReLU

784 $\rightarrow$ 1024

Linear + LeakyReLU

1024 $\rightarrow$ 512

Linear + LeakyReLU

512 $\rightarrow$ 256

Linear (pas de σ !)

256 $\rightarrow$ 1

Pourquoi pas de sigmoid sur D ?

Dans WGAN, D est un potentiel Kantorovich, pas une probabilité. On utilise D(x) directement comme énergie $\rightarrow$ cohérent avec $E(z) = \frac{1}{2}\|z\|^2 - D(G(z))$.

DDLS avec énergie latente & cosine annealing

Fonction d'énergie latente implémentée (cas WGAN)

$$E(z) = \frac{1}{2}\|z\|^2 - D(G(z))$$

Boucle Langevin (generate.py)

```
for i in range(steps):
    cos = 0.5*(1+cos(π·i/steps))
    ε = ε_min+(ε_0-ε_min)*cos

    z.requires_grad_(True)
    x = G(z)
    E = 0.5*z2.sum() - D(x).sum()

    grad_z = autograd.grad(E, z)[0]

    with torch.no_grad():
        noise = randn_like(z)*σ
        z = z-(ε/2)*grad_z+sqrt(ε)*noise
        z = z.detach()
```

Notre ajout : Cosine annealing du pas ε

$$\epsilon(i) = \epsilon_{\min} + (\epsilon_0 - \epsilon_{\min}) \cdot (1 + \cos(\pi \cdot i / N)) / 2$$

Début (i=0)

ε = 0.005

Exploration large

Milieu

Transition douce

Fin (i=N)

ε = 1e-6

Affinement précis

Hyperparamètres DDLS : steps = 1000 · ε₀ = 0.005 · ε_{min} = 1e-6 · σ = 0.1 · batch = 2048 · z = torch.randn(batch,100)

Résultats / Comparaison des méthodes

Méthode	FID ↓	Precision ↑	Recall ↑
Gan original	44.64	0.5604	0.5940
Hard Truncation ($\psi=2$)	43.87	0.5650	0.5970
Soft Truncation ($\psi=2$)	44.47	0.5520	0.6028★
DDLs	33.46★	0.6354★	0.2584

Interprétation des résultats DDLS

33.46

FID

-25.2%

vs 44.64 pour le GAN

0.635

Precision

+13.4%

vs 0.560 pour le GAN

0.258

Recall

-56.5%

vs 0.594 pour le GAN

Ce qui fonctionne

- FID amélioré de 25% → meilleure qualité globale des images
- Precision +13% → les samples générés ressemblent plus aux vraies données
- Valide le théorème : DDLS corrige les biais du générateur
- Applicable sans re-entraînement sur un WGAN-GP

Les limitations

- Recall chute à 0.258 → fort mode dropping résiduel
- DDLS concentre sur les modes haute énergie → perd la diversité
- Truncation (hard/soft) ne change quasi rien ici : GAN déjà stable

Pourquoi ce trade-off Precision / Recall ?

La dynamique de Langevin minimise $E(z) = \frac{1}{2}\|z\|^2 - D(G(z))$. Elle converge vers des z qui maximisent $D(G(z))$, c'est-à-dire des images très vraisemblables. Cela augmente la précision mais réduit l'exploration des modes → recall bas. C'est le pendant du mode dropping prédit par le papier.

Conclusion

Un GAN entraîné définit implicitement un EBM, exploitable post-hoc sans re-entraînement.

Fondement théorique

Théorème prouvé : sous discriminateur optimal, $p_{d^*} = p_d$ exactement.

Efficacité pratique

Langevin dans l'espace latent (faible dim.) → mélange beaucoup plus rapide.

Plug-and-play

S'applique à tout GAN pré-entraîné. Pas de paramètre additionnel.

Limite principale

Mode dropping hérité du générateur. DDLS ne peut récupérer un mode inexistant.